

ПРАКТИЧЕСКАЯ РАБОТА

Тема: Создание таблиц и базы данных

Дисциплина: ОП.06 «Основы проектирования баз данных»

1. Цель работы

Получить практические навыки:

- создания базы данных;
- создания таблиц различными способами;
- определения типов данных;
- задания ограничений целостности;
- использования первичных и внешних ключей;
- установки автоинкремента;
- создания связей между таблицами;
- настройки каскадного обновления и удаления;
- создания схемы данных;
- заполнения таблиц;
- написания и сохранения SQL-скриптов.

2. Постановка задачи

Необходимо создать базу данных информационной системы «Колледж».

База данных должна хранить информацию:

- об учебных группах;
- о студентах;
- о преподавателях;
- о дисциплинах;
- об учебных занятиях.

Необходимо организовать связи между таблицами и обеспечить целостность данных.

3. Структура базы данных

В базе данных необходимо создать следующие таблицы:

Таблица «Группы»

Поле	Тип	Назначение
group_id	INT	Код группы
group_name	VARCHAR(20)	Название группы
specialty	VARCHAR(100)	Специальность
admission_year	YEAR	Год поступления

Таблица «Студенты»

Поле	Тип	Назначение
student_id	INT	Код студента
last_name	VARCHAR(50)	Фамилия
first_name	VARCHAR(50)	Имя
middle_name	VARCHAR(50)	Отчество
birth_date	DATE	Дата рождения
group_id	INT	Код группы

Таблица «Преподаватели»

Поле	Тип	Назначение
teacher_id	INT	Код преподавателя
last_name	VARCHAR(50)	Фамилия
first_name	VARCHAR(50)	Имя
department	VARCHAR(100)	Кафедра

Таблица «Дисциплины»

Поле	Тип	Назначение
discipline_id	INT	Код дисциплины
discipline_name	VARCHAR(100)	Название дисциплины
hours	INT	Количество часов

Таблица «Занятия»

Поле	Тип	Назначение
lesson_id	INT	Код занятия
group_id	INT	Код группы
discipline_id	INT	Код дисциплины
teacher_id	INT	Код преподавателя
lesson_date	DATE	Дата занятия
lesson_type	VARCHAR(30)	Вид занятия

Создание пользователя:

```
Query  Query History
1  CREATE USER ivanov WITH PASSWORD 'secret1';
2  GRANT ALL PRIVILEGES ON DATABASE student_ivanov TO ivanov;
3
```

Создание базы данных

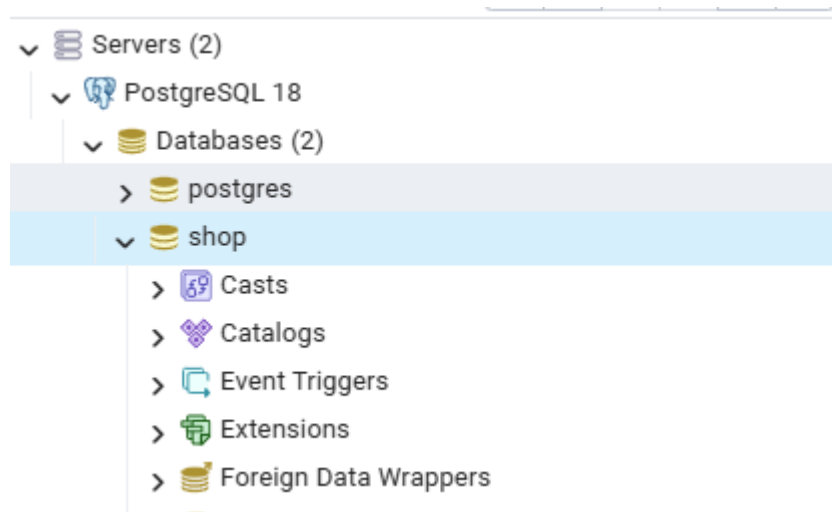
Создайте новую базу данных с названием:

Ivanov_bd (указываете вашу фамилию)

SQL-команда:

```
Query  Query History
1  CREATE DATABASE shop ENCODING = 'UTF8';
```

Выберите созданную базу данных:



Проверьте наличие базы данных.

Создание таблицы «Группы»

Создайте таблицу «Группы».

Поле group_id должно:

- быть первичным ключом;
- автоматически увеличиваться;
- не допускать NULL.

Query Query History

```
1 CREATE TABLE groups (  
2     group_id SERIAL PRIMARY KEY,  
3     group_name VARCHAR(20) NOT NULL UNIQUE,  
4     specialty VARCHAR(100) NOT NULL,  
5     admission_year INT NOT NULL  
6 );
```

Обратите внимание на:

AUTO_INCREMENT — автоматически формирует следующий номер записи.

PRIMARY KEY — определяет первичный ключ.

NOT NULL — запрещает отсутствие значения.

UNIQUE — не позволяет повторять название группы.

Создание таблицы «Студенты»

Создайте таблицу студентов.

Query	Query History
1	CREATE TABLE students (2 student_id SERIAL PRIMARY KEY , 3 last_name VARCHAR(50) NOT NULL , 4 first_name VARCHAR(50) NOT NULL , 5 middle_name VARCHAR(50) , -- NULL разрешён 6 birth_date DATE NOT NULL , 7 group_id INTEGER NOT NULL , 8 9 CONSTRAINT fk_students_groups 10 FOREIGN KEY (group_id) 11 REFERENCES groups(group_id) 12 ON UPDATE CASCADE 13 ON DELETE CASCADE 14);

Здесь появляется **внешний ключ**.

Поле:

students.group_id

ссылается на:

groups.group_id

Таким образом, устанавливается связь между студентом и его группой.

Создание остальных таблиц

Создайте таблицу преподавателей:

Query Query History

```
1 CREATE TABLE teachers (  
2     teacher_id SERIAL PRIMARY KEY,  
3     last_name VARCHAR(50) NOT NULL,  
4     first_name VARCHAR(50) NOT NULL,  
5     department VARCHAR(100) NOT NULL  
6 );
```

Создайте таблицу дисциплин:

Query Query History

```
1 CREATE TABLE disciplines (  
2     discipline_id SERIAL PRIMARY KEY,  
3     discipline_name VARCHAR(100) NOT NULL UNIQUE,  
4     hours INTEGER NOT NULL CHECK (hours > 0)  
5 );
```

Создайте таблицу занятий:

Query	Query History
1	CREATE TABLE lessons (
2	lesson_id SERIAL PRIMARY KEY ,
3	group_id INTEGER NOT NULL ,
4	discipline_id INTEGER NOT NULL ,
5	teacher_id INTEGER NOT NULL ,
6	lesson_date DATE NOT NULL ,
7	lesson_type VARCHAR(30) NOT NULL ,
8	
9	CONSTRAINT fk_lessons_groups
10	FOREIGN KEY (group_id)
11	REFERENCES groups(group_id)
12	ON UPDATE CASCADE
13	ON DELETE CASCADE ,
14	
15	CONSTRAINT fk_lessons_disciplines
16	FOREIGN KEY (discipline_id)
17	REFERENCES disciplines(discipline_id)
18	ON UPDATE CASCADE
19	ON DELETE RESTRICT ,
20	
21	CONSTRAINT fk_lessons_teachers
22	FOREIGN KEY (teacher_id)
23	REFERENCES teachers(teacher_id)
24	ON UPDATE CASCADE
25	ON DELETE RESTRICT
26	);

Изучение ограничений целостности

Проанализируйте созданные таблицы.

В работе используются следующие ограничения:

PRIMARY KEY

Первичный ключ однозначно идентифицирует запись.

Например:

```
student_id INT AUTO_INCREMENT PRIMARY KEY
```

FOREIGN KEY

Внешний ключ устанавливает связь между таблицами.

Например:

FOREIGN KEY (group_id)
REFERENCES groups(group_id)

NOT NULL

Запрещает отсутствие значения:

last_name VARCHAR(50) **NOT NULL**

UNIQUE

Запрещает повторение значения:

group_name VARCHAR(20) **UNIQUE**

CHECK

Позволяет задать условие:

hours INT **CHECK** (hours > 0)

Автоинкремент

Проверьте работу автоматической нумерации.

Добавьте несколько групп:

Query	Query History
1	INSERT INTO groups
2	(group_name, specialty, admission_year)
3	VALUES
4	('П-21', 'Программирование', 2026),
5	('ИС-22', 'Информационные системы', 2026),
6	('ЭК-21', 'Экономика', 2026);
7	

Обратите внимание: значение group_id мы не указывали.

СУБД сформировала его автоматически.

Проверьте результат:

Query

Query History

1

2

SELECT * FROM groups;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

	group_id [PK] integer	group_name character varying (20)	specialty character varying (100)	admission_year integer
1	1	П-21	Программирование	2026
2	2	ИС-22	Информационные систе...	2026
3	3	ЭК-21	Экономика	2026

Заполнение таблиц

Заполните таблицу студентов:

Query	Query History
1	INSERT INTO students
2	(last_name, first_name, middle_name, birth_date, group_id)
3	VALUES
4	('Иванов', 'Иван', 'Иванович', '2007-03-15', 1),
5	('Петров', 'Алексей', 'Сергеевич', '2007-07-21', 1),
6	('Сидорова', 'Анна', 'Игоревна', '2006-11-04', 2),
7	('Кузнецов', 'Максим', 'Олегович', '2007-01-18', 2),
8	('Смирнова', 'Елена', 'Андреевна', '2006-09-27', 3);
9	

Заполните таблицу преподавателей:

Query	Query History
1	INSERT INTO teachers
2	(last_name, first_name, department)
3	VALUES
4	('Волков', 'Александр', 'Информационные технологии'),
5	('Орлова', 'Мария', 'Общеобразовательные дисциплины'),
6	('Фёдоров', 'Дмитрий', 'Программирование');
7	
8	

Заполните таблицу дисциплин:

Query	Query History
1	INSERT INTO disciplines
2	(discipline_name, hours)
3	VALUES
4	('Основы проектирования баз данных', 72),
5	('Информационные технологии', 72),
6	('Программирование', 144),
7	('Компьютерные сети', 72);
8	
9	

Заполните таблицу занятий:

Query	Query History
1	INSERT INTO lessons
2	(group_id, discipline_id, teacher_id, lesson_date, lesson_type)
3	VALUES
4	(1, 1, 1, '2026-09-10', 'Лекция'),
5	(1, 3, 3, '2026-09-11', 'Практика'),
6	(2, 1, 1, '2026-09-10', 'Практика'),
7	(2, 2, 2, '2026-09-12', 'Лекция'),
8	(3, 2, 2, '2026-09-13', 'Практика');
9	
10	
11	

Проверка ссылочной целостности

Попробуйте выполнить команду:

Query	Query History
1	INSERT INTO students
2	(last_name, first_name, birth_date, group_id)
3	VALUES
4	('Тестов', 'Иван', '2007-05-10', 999);
5	
6	
7	
8	

Data Output Messages Notifications

ERROR: INSERT или UPDATE в таблице "students" нарушает ограничение внешнего ключа "fk_students_groups"
Ключ (group_id)=(999) отсутствует в таблице "groups".

ОШИБКА: INSERT или UPDATE в таблице "students" нарушает ограничение внешнего ключа "fk_students_groups"
SQL state: 23503
Detail: Ключ (group_id)=(999) отсутствует в таблице "groups".

Операция должна завершиться ошибкой.

Почему?

Потому что группы с кодом 999 не существует.

Это пример работы **ссылочной целостности**.

Внешний ключ не позволяет создать ссылку на несуществующую запись.

Каскадное обновление

В таблице students используется:

ON UPDATE CASCADE

Это означает, что изменение ключевого значения в родительской таблице может автоматически распространиться на связанные записи.

Проверьте работу механизма.

Например, измените идентификатор группы:

Query	Query History
1	UPDATE groups
2	SET group_id = 10
3	WHERE group_id = 1;
4	
5	
6	
7	

Затем выполните:

Query Query History

```
1 SELECT * FROM students;
```

Data Output Messages Notifications

Showing rows: 1 to 5

	student_id [PK] integer	last_name character varying (50)	first_name character varying (50)	middle_name character varying (50)	birth_date date	group_id integer
1	3	Сидорова	Анна	Игоревна	2006-11-04	2
2	4	Кузнецов	Максим	Олегович	2007-01-18	2
3	5	Смирнова	Елена	Андреевна	2006-09-27	3
4	1	Иванов	Иван	Иванович	2007-03-15	

✓ Successfully run.

Связанные значения group_id у студентов должны измениться автоматически.

После проверки можно вернуть значение обратно:

Query Query History

```
1 UPDATE groups
2 SET group_id = 1
3 WHERE group_id = 10;
```

Каскадное удаление

Для связи группы и студентов используется:

ON DELETE CASCADE

Это означает, что при удалении группы связанные с ней записи студентов также будут удалены.

Для проверки создайте отдельную тестовую группу:

Query	Query History
1	INSERT INTO groups
2	(group_name, specialty, admission_year)
3	VALUES
4	('ТЕСТ-01', 'Тестовая специальность', 2026);
5	
6	
7	
8	
9	
10	

Запомните автоматически созданный group_id.

Добавьте студента в эту группу:

Query	Query History
1	INSERT INTO students
2	(last_name, first_name, birth_date, group_id)
3	VALUES
4	('Тестовый', 'Студент', '2007-01-01', <ID_ГРУППЫ>);
5	
6	
7	
8	
9	

Удалите тестовую группу:

Query	Query History
1	DELETE FROM groups
2	WHERE group_id = <ID_ГРУППЫ>;
3	
4	
5	
6	
7	
8	
9	

Проверьте таблицу студентов.

Запись тестового студента должна быть удалена автоматически.

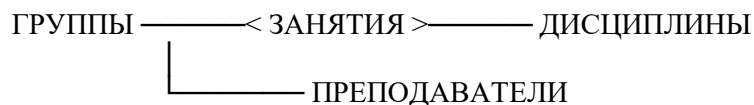
Внимание: каскадное удаление следует применять осторожно, поскольку удаление родительской записи может привести к удалению большого количества связанных данных.

Создание схемы данных

Используя графический инструмент СУБД, создайте схему базы данных.

На схеме должны быть представлены:

ГРУППЫ



Обозначение:

1 : N

означает связь «один ко многим».

Например:

одна группа → много студентов.

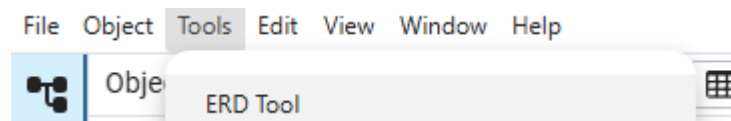
Аналогично:

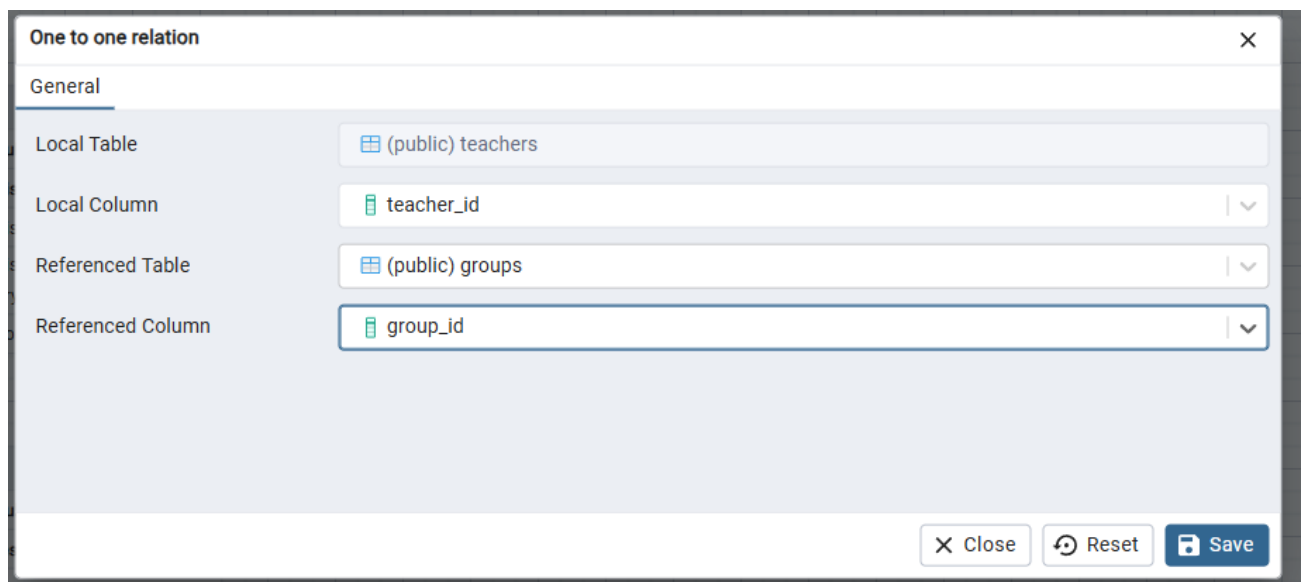
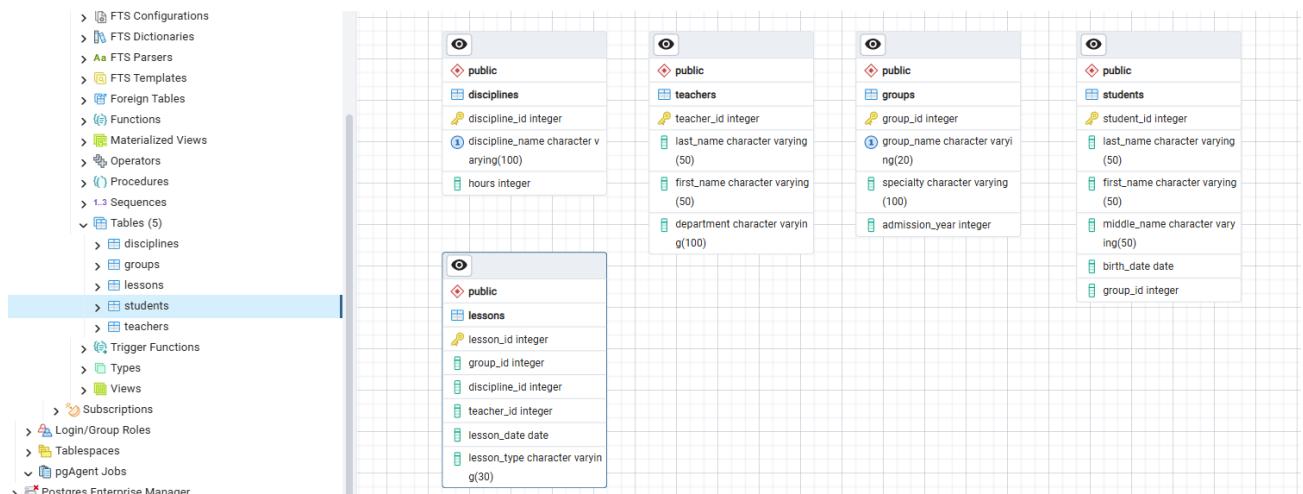
одна группа → много занятий.

одна дисциплина → много занятий.

один преподаватель → много занятий.

Создайте схему и сохраните её в проекте.





Заполнение таблиц различными способами

Используйте как минимум два способа заполнения таблиц.

Способ 1 — команда INSERT

Например:

```
Query  Query History
1  INSERT INTO groups
2      (group_name, specialty, admission_year)
3  VALUES
4      ('П-23', 'Программирование', 2026);
5
6
7
8
9
10
```

Откройте таблицу и добавьте несколько записей через интерфейс.

После этого выполните:

```
Query  Query History
1  SELECT * FROM groups;
2
3
4
5
6
7
8
```

Data Output Messages Notifications				
      SQL				
	group_id [PK] integer 	group_name character varying (20) 	specialty character varying (100) 	admission_year integer 
1	2	ИС-22	Информационные сис...	2026
2	3	ЭК-21	Экономика	2026
3	1	П-21	Программирование	2026
4	4	ТЕСТ-01	Тестовая специальнос...	2026

Сравните результаты.

Проверка структуры таблиц

Для каждой таблицы изучите её структуру.

Query

Query History

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

SELECT

column_name,

data_type,

is_nullable,

column_default

FROM information_schema.columns

WHERE table_name = 'groups'

ORDER BY ordinal_position;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

Sh

	column_name name	data_type character varying	is_nullable character varying (3)	column_default character varying
1	group_id	integer	NO	nextval('groups_group_id_seq'::regcla...
2	group_name	character varying	NO	[null]
3	specialty	character varying	NO	[null]
4	admission_year	integer	NO	[null]

Определите:

1. Какие поля являются первичными ключами?
2. Какие поля являются внешними ключами?
3. Где используется AUTO_INCREMENT?
4. Где используется NOT NULL?
5. Где используется UNIQUE?
6. Где используется CHECK?
7. Какие таблицы связаны между собой?

Создание и сохранение SQL-скрипта

Создайте единый SQL-файл:

college_db.sql

В него необходимо включить команды:

1. создания базы данных;
2. выбора базы данных;
3. создания таблиц;
4. создания ограничений;
5. создания внешних ключей;
6. заполнения таблиц;
7. нескольких тестовых запросов.

Скрипт должен позволять восстановить структуру и начальные данные базы данных.

Query	Query History
1	-- =====
2	-- 1. Создание базы данных
3	-- =====
4	
5	-- Удаляем базу данных, если она уже существует (с осторожностью!)
6	-- DROP DATABASE IF EXISTS college_db;
7	
8	CREATE DATABASE college_db
9	ENCODING = 'UTF8'
10	LC_COLLATE = 'en_US.UTF-8'
11	LC_CTYPE = 'en_US.UTF-8'
12	CONNECTION LIMIT = 100;
13	
14	-- =====
15	-- 2. Подключение к базе данных
16	-- =====
17	

```

18 -- В psql используется команда: \c college_db
19 -- В скрипте эта команда не выполняется, подключение нужно делать отдельно.
20
21 -- =====
22 -- 3. Создание таблиц
23 -- =====
24
25 -- Таблица "Группы"
26 CREATE TABLE IF NOT EXISTS groups (
27     group_id SERIAL PRIMARY KEY,
28     group_name VARCHAR(20) NOT NULL UNIQUE,
29     specialty VARCHAR(100) NOT NULL,
30     admission_year INTEGER NOT NULL
31 );
32
33 -- Таблица "Студенты"
34 CREATE TABLE IF NOT EXISTS students (
35     student_id SERIAL PRIMARY KEY,
36     last_name VARCHAR(50) NOT NULL,
37     first_name VARCHAR(50) NOT NULL,
38     middle_name VARCHAR(50),
39     birth_date DATE NOT NULL,
40     group_id INTEGER NOT NULL,
41
42     CONSTRAINT fk_students_groups
43         FOREIGN KEY (group_id)
44         REFERENCES groups(group_id)
45         ON UPDATE CASCADE
46         ON DELETE CASCADE
47 );
48
49 -- Таблица "Преподаватели"
50 CREATE TABLE IF NOT EXISTS teachers (
51     teacher_id SERIAL PRIMARY KEY,
52     last_name VARCHAR(50) NOT NULL,
53     first_name VARCHAR(50) NOT NULL,
54     department VARCHAR(100) NOT NULL
55 );
56

```

```
57  -- Таблица "Дисциплины"
58  CREATE TABLE IF NOT EXISTS disciplines (
59      discipline_id SERIAL PRIMARY KEY,
60      discipline_name VARCHAR(100) NOT NULL UNIQUE,
61      hours INTEGER NOT NULL CHECK (hours > 0)
62  );
63
64  -- Таблица "Занятия"
65  CREATE TABLE IF NOT EXISTS lessons (
66      lesson_id SERIAL PRIMARY KEY,
67      group_id INTEGER NOT NULL,
68      discipline_id INTEGER NOT NULL,
69      teacher_id INTEGER NOT NULL,
70      lesson_date DATE NOT NULL,
71      lesson_type VARCHAR(30) NOT NULL,
72
73      CONSTRAINT fk_lessons_groups
74          FOREIGN KEY (group_id)
75          REFERENCES groups(group_id)
76          ON UPDATE CASCADE
77          ON DELETE CASCADE,
78
79      CONSTRAINT fk_lessons_disciplines
80          FOREIGN KEY (discipline_id)
81          REFERENCES disciplines(discipline_id)
82          ON UPDATE CASCADE
      -----
```

```
83         ON DELETE RESTRICT,
84
85     CONSTRAINT fk_lessons_teachers
86         FOREIGN KEY (teacher_id)
87         REFERENCES teachers(teacher_id)
88         ON UPDATE CASCADE
89         ON DELETE RESTRICT
90 );
91
92 -- Дополнительная таблица "Оценки" (для дополнительного задания)
93 CREATE TABLE IF NOT EXISTS grades (
94     grade_id SERIAL PRIMARY KEY,
95     student_id INTEGER NOT NULL,
96     discipline_id INTEGER NOT NULL,
97     grade INTEGER NOT NULL CHECK (grade BETWEEN 2 AND 5),
98     grade_date DATE NOT NULL,
99
100     CONSTRAINT fk_grades_students
101         FOREIGN KEY (student_id)
102         REFERENCES students(student_id)
103         ON UPDATE CASCADE
104         ON DELETE CASCADE,
105
106     CONSTRAINT fk_grades_disciplines
107         FOREIGN KEY (discipline_id)
108         REFERENCES disciplines(discipline_id)
```

```

109         ON UPDATE CASCADE
110         ON DELETE RESTRICT
111     );
112
113     -- =====
114     -- 4. Заполнение таблиц данными
115     -- =====
116
117     -- Группы
118     INSERT INTO groups (group_name, specialty, admission_year) VALUES
119         ('П-21', 'Программирование', 2026),
120         ('ИС-22', 'Информационные системы', 2026),
121         ('ЭК-21', 'Экономика', 2026);
122
123     -- Студенты
124     INSERT INTO students (last_name, first_name, middle_name, birth_date, group_id) VALUES
125         ('Иванов', 'Иван', 'Иванович', '2007-03-15', 1),
126         ('Петров', 'Алексей', 'Сергеевич', '2007-07-21', 1),
127         ('Сидорова', 'Анна', 'Игоревна', '2006-11-04', 2),
128         ('Кузнецов', 'Максим', 'Олегович', '2007-01-18', 2),
129         ('Смирнова', 'Елена', 'Андреевна', '2006-09-27', 3);
130
131     -- Преподаватели
132     INSERT INTO teachers (last_name, first_name, department) VALUES
133         ('Волков', 'Александр', 'Информационные технологии'),
134         ('Орлова', 'Мария', 'Общеобразовательные дисциплины'),
135         ('Фёдоров', 'Дмитрий', 'Программирование');
136
137     -- Дисциплины
138     INSERT INTO disciplines (discipline_name, hours) VALUES
139         ('Основы проектирования баз данных', 72),
140         ('Информационные технологии', 72),
141         ('Программирование', 144),
142         ('Компьютерные сети', 72);
143
144     -- Занятия
145     INSERT INTO lessons (group_id, discipline_id, teacher_id, lesson_date, lesson_type) VALUES
146         (1, 1, 1, '2026-09-10', 'Лекция'),
147         (1, 3, 3, '2026-09-11', 'Практика'),
148         (2, 1, 1, '2026-09-10', 'Практика'),
149         (2, 2, 2, '2026-09-12', 'Лекция'),
150         (3, 2, 2, '2026-09-13', 'Практика');
151
152     -- =====
153     -- 5. Тестовые запросы
154     -- =====
155
156     -- Список всех студентов с названиями групп
157     SELECT
158         s.student_id,
159         s.last_name || ' ' || s.first_name AS full_name,

```

```
160         g.group_name,
161         g.specialty
162 FROM students s
163 JOIN groups g ON s.group_id = g.group_id
164 ORDER BY g.group_name, s.last_name;
165
166 -- Количество студентов в каждой группе
167 SELECT
168     g.group_name,
169     COUNT(s.student_id) AS student_count
170 FROM groups g
171 LEFT JOIN students s ON g.group_id = s.group_id
172 GROUP BY g.group_id, g.group_name
173 ORDER BY g.group_name;
174
175 -- Расписание занятий (дата, группа, дисциплина, преподаватель, вид занятия)
176 SELECT
177     l.lesson_date,
178     g.group_name,
179     d.discipline_name,
180     t.last_name || ' ' || t.first_name AS teacher_name,
181     l.lesson_type
182 FROM lessons l
183 JOIN groups g ON l.group_id = g.group_id
184 JOIN disciplines d ON l.discipline_id = d.discipline_id
185 JOIN teachers t ON l.teacher_id = t.teacher_id
```

```

185 JOIN teachers t ON l.teacher_id = t.teacher_id
186 ORDER BY l.lesson_date;
187
188 -- Список дисциплин с количеством часов
189 SELECT
190     discipline_name,
191     hours
192 FROM disciplines
193 ORDER BY hours DESC;
194
195 -- =====
196 -- 6. Дополнительное задание (оценки)
197 -- =====
198
199 -- Добавляем оценки для студентов
200 INSERT INTO grades (student_id, discipline_id, grade, grade_date) VALUES
201     (1, 1, 5, '2026-09-15'),
202     (1, 3, 4, '2026-09-16'),
203     (2, 1, 3, '2026-09-15'),
204     (2, 3, 5, '2026-09-16'),
205     (3, 1, 4, '2026-09-17'),
206     (3, 2, 5, '2026-09-18'),
207     (4, 2, 4, '2026-09-18'),
208     (4, 1, 3, '2026-09-19'),
209     (5, 2, 5, '2026-09-20'),
210     (5, 1, 4, '2026-09-21');
211
212 -- Средний балл по дисциплинам
213 SELECT
214     d.discipline_name,
215     AVG(g.grade) AS average_grade
216 FROM grades g
217 JOIN disciplines d ON g.discipline_id = d.discipline_id
218 GROUP BY d.discipline_name
219 ORDER BY average_grade DESC;

```

Домашнее задание

Создайте таблицу «Оценки»:

Поле	Назначение
grade_id	Код оценки
student_id	Код студента
discipline_id	Код дисциплины
grade	Оценка
grade_date	Дата

Требования:

- grade_id — первичный ключ;
- используется автоинкремент;
- student_id — внешний ключ;
- discipline_id — внешний ключ;
- оценка должна находиться в диапазоне от 2 до 5;
- дата оценки обязательна.

Пример:

```
Query  Query History
1  CREATE TABLE grades (
2      grade_id INT AUTO_INCREMENT PRIMARY KEY,
3      student_id INT NOT NULL,
4      discipline_id INT NOT NULL,
5      grade INT NOT NULL CHECK (grade BETWEEN 2 AND 5),
6      grade_date DATE NOT NULL,
7
8      CONSTRAINT fk_grades_students
9          FOREIGN KEY (student_id)
10             REFERENCES students(student_id)
11             ON UPDATE CASCADE
12             ON DELETE CASCADE,
13
14     CONSTRAINT fk_grades_disciplines
15         FOREIGN KEY (discipline_id)
16             REFERENCES disciplines(discipline_id)
17             ON UPDATE CASCADE
18             ON DELETE RESTRICT
19 );
```

Добавьте в таблицу не менее **10 оценок**.

Что должно быть представлено преподавателю

По результатам практической работы студент предоставляет:

1. Скриншот созданной базы данных.
2. Скриншоты структуры таблиц.
3. Скриншот схемы данных со связями.
4. Скриншот заполненных таблиц.
5. Результат проверки ссылочной целостности.
6. Результат проверки каскадного обновления.
7. Результат проверки каскадного удаления.
8. Файл college_db.sql.
9. Ответы на контрольные вопросы.